FitTrack Personalized Fitness Recommendations

Introduction

Project title: AI FitTrack
Team ID: 6ab4bdeea238dc999eb52bbe
Team Leader: Tanish M
Mail ID: ncas2427sz1325@ncas.in
NM ID: 484EBA8C921BB3FBB3941B00B952FD17

Team members:

1. Thamim Ansari Masthan. A
   Mail ID: ncas2427sz1309@ncas.in
   NM ID: 2C8CD6ECA8AEDB3983B4AE59A88748E5

2. Udayakumar. S
   Mail ID: ncas2427sz1197@ncas.in
   NM ID: F60CE6EA07A090796E94FF64F4D4BCE4

3. Sakthivel. M
   Mail ID: ncas2427sz1144@ncas.in
   NM ID: 3CD03B22A675251BBC2426D956A7CFF6

4. Rohan C. P.
   Mail ID: ncas2427sz1100@ncas.in
   NM ID: 1DB44059DCD1D719AEF45F58B192BABD

Description

FitSense is a secure REST API for workout tracking and personalized fitness guidance. Built with Node.js, Express, MongoDB and Mongoose, it supports account authentication, workout management, search, and AI recommendations through Google Gemini.

Scenario-Based Case Study

Background

Rahul tracks workouts in notebooks but struggles to organize his history, measure progress, and choose exercises suited to his goals.

Problem

Solution

FitSense centralizes workout records and provides authenticated APIs to create, view, search, update and delete them. Gemini uses a user's age, goal and experience to generate recommendations and fitness insights.

JWT authentication, bcrypt password hashing, input validation and centralized error handling protect user data.

Usage

User Usage

Users register and sign in to receive a JWT, then use protected endpoints to manage workouts and request AI guidance.

Using authenticated APIs, users can:

- Request personalized workout recommendations and fitness insights.
- Obtain AI-generated fitness insights based on workout statistics.

AI Usage

The Gemini service uses age, fitness goal and experience level to create a workout plan, weekly suggestions, safety tips and motivation. It can also summarize workout totals, duration and calories.

Users provide information such as:

Gemini processes this information and returns:

Similarly, users can submit workout statistics including:

The AI module analyzes these values and produces personalized fitness insights.

Outcome

FitSense replaces manual workout logs with secure tracking and AI-supported recommendations. Its modular design can later support nutrition, wearable devices, reminders and analytics.

1. Software Requirements

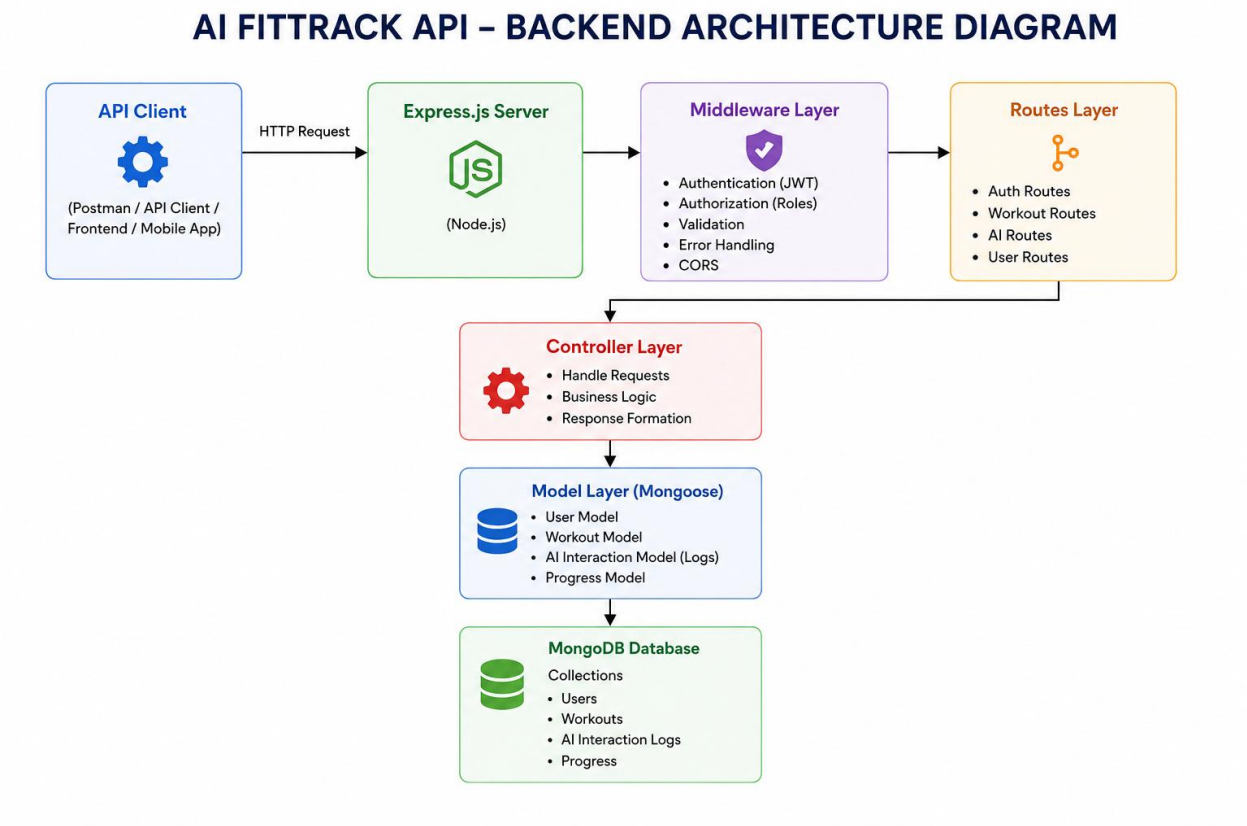
Windows, macOS or Linux; Node.js 16+, npm 8+, Express.js, MongoDB, Postman or Thunder Client, and a code editor.

- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

2. Hardware Requirements

- Intel Core i5 (8th generation) / AMD Ryzen 5 or equivalent; 8 GB RAM minimum (16 GB recommended); 1 GB available storage.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Epic 1: Project Architecture



FitSense uses an MVC-style backend. Requests pass from the client through Express routes and JWT middleware to controllers, services, MongoDB models or the Gemini service. Responses return as JSON.

The route, controller, service and data layers separate responsibilities and support maintenance and extension.

This layered approach improves modularity, security, and maintainability while ensuring that each component has a single responsibility.

Overall Architecture Components

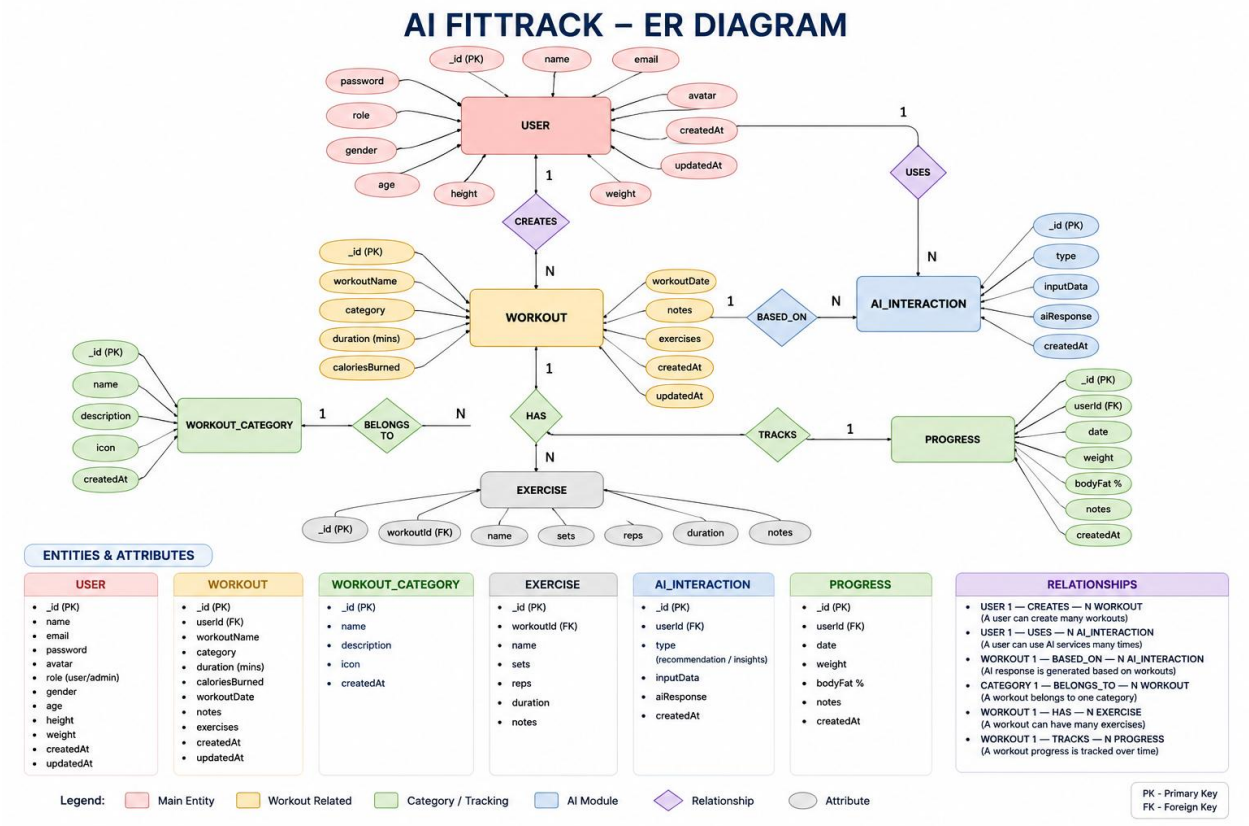
Each component communicates through well-defined interfaces, ensuring loose coupling and easier future enhancements.

Components Definition

The backend separates client access, routing, authentication, controllers, services and data storage. Each layer handles a focused part of a request.

The data model has two core entities: User and Workout. One user can own many workouts; each workout belongs to one user.

The AI FitTrack backend consists of two primary collections that are logically connected through a one-to-many relationship.



1. User Entity

Stores account and profile information, including a unique email and hashed password.

Field	Type	Description
_id	ObjectId	Primary Key
name	String	User Name
email	String	Unique Email Address
password	String	Encrypted Password
createdAt	Date	Record Creation Time
updatedAt	Date	Last Updated Time

2. Workout Entity

Stores workout details and a reference to the owning user.

Field	Type	Description
_id	ObjectId	Primary Key
user	ObjectId	References User Collection
workoutName	String	Workout Name
category	String	Workout Category
duration	Number	Duration (Minutes)
caloriesBurned	Number	Calories Burned
workoutDate	Date	Workout Date

createdAt	Date	Created Timestamp
updatedAt	Date	Updated Timestamp

Relationship Type: One-to-Many

One user can create multiple workout records; each record belongs to one authenticated user.

This relationship is maintained using the **user** field inside the Workout schema, which stores the ObjectId of the corresponding user.

Key Features

Core capabilities include secure authentication, workout CRUD and search, Gemini recommendations and insights, RESTful JSON APIs, and centralized error handling.

1. Secure Authentication

2. Workout Management

Each workout is linked to its authenticated user.

4. Google Gemini Integration

The backend communicates with Google Gemini AI to generate intelligent workout recommendations.

5. AI Fitness Insights

The AI module analyzes workout statistics.

6. Centralized Error Handling

7. RESTful API Design

8. Modular Architecture

Roles and Responsibilities

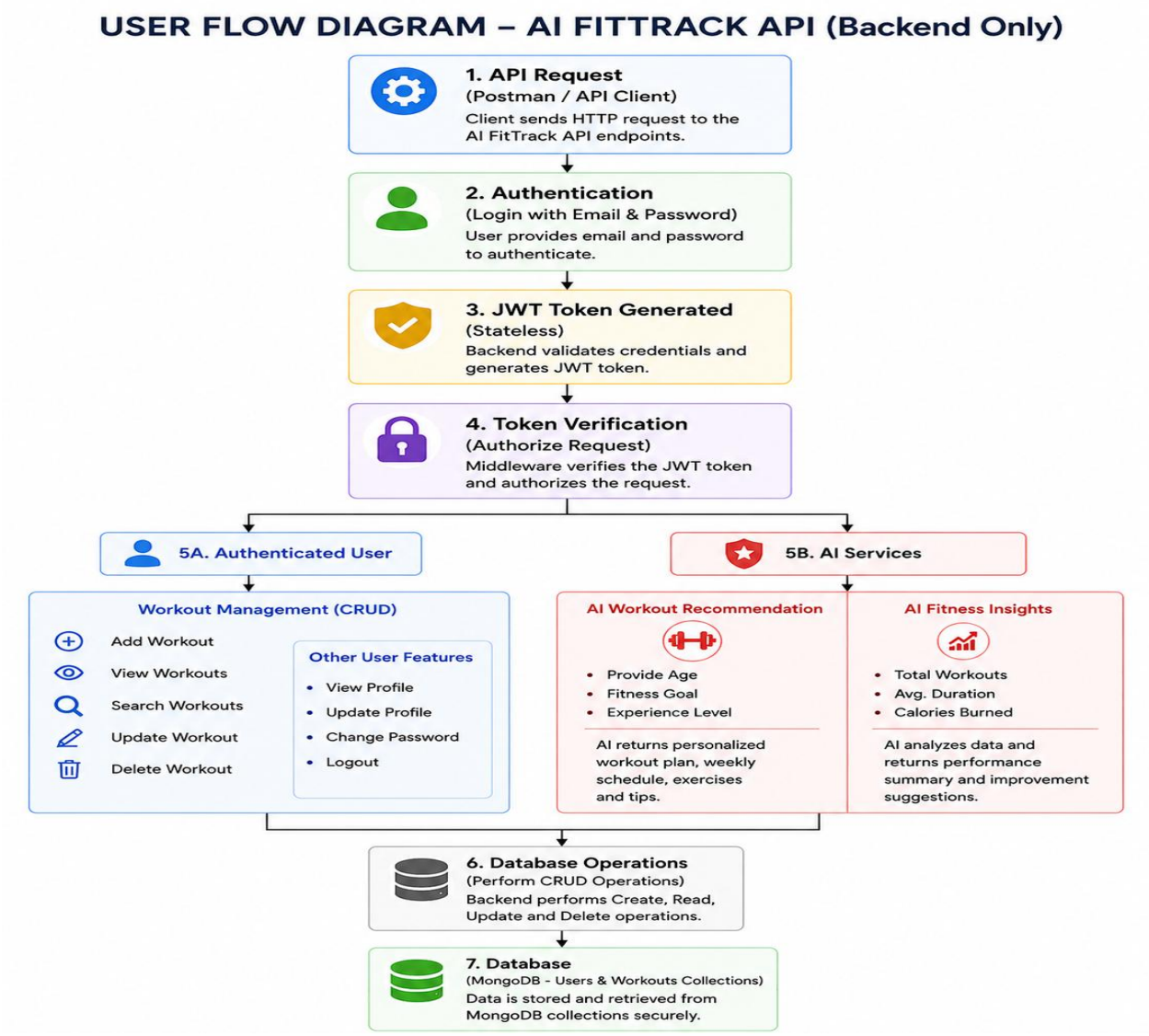
The AI FitTrack backend supports one primary role.

`Authenticated User`

Permissions

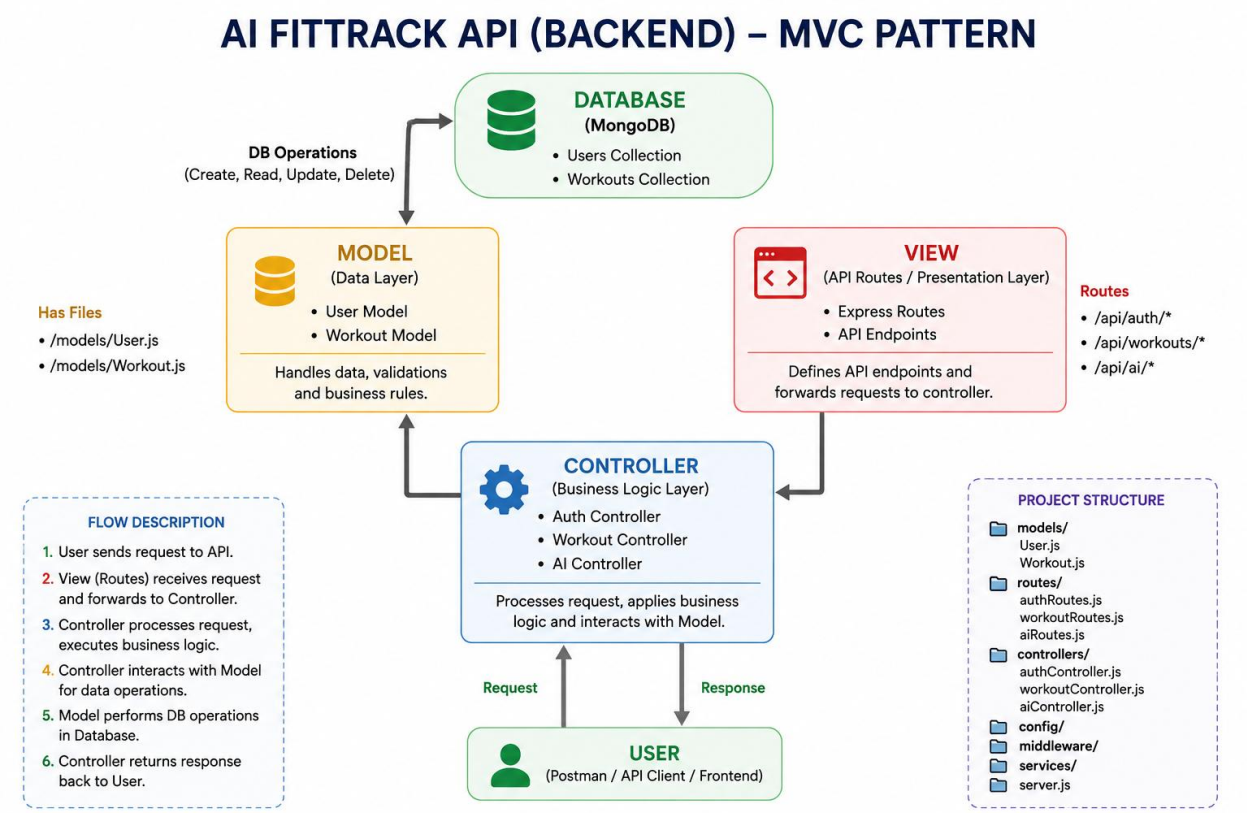
Operation	Permission
Register	Yes
Login	Yes
View Profile	Yes
Add Workout	Yes
Get Workouts	Yes
Search Workouts	Yes
Update Workout	Yes
Delete Workout	Yes
AI Recommendation	Yes
Fitness Insights	Yes

User Flow



MVC Architecture Pattern

The FitTrack AI is designed using the **Model–View–Controller (MVC)** architecture pattern. This architecture separates application logic into independent components, improving code organization, maintainability, and scalability.



Model Layer (Data Layer)

The Model Layer defines the structure of the application's data using Mongoose schemas. It communicates directly with MongoDB and performs database operations.

View Layer (API Interface)

Controller Layer (Business Logic)

The Controller Layer acts as the intermediary between the View and Model. It processes incoming requests, validates input, invokes services and models, and sends structured JSON responses back to the client.

The separation of these three layers ensures loose coupling, simplifies testing, enhances code readability, and makes the application easier to extend with future features.

Epic 2: Project Execution

· Install ThunderClient (VS Code extension).

· Start the backend server:

“npm start”

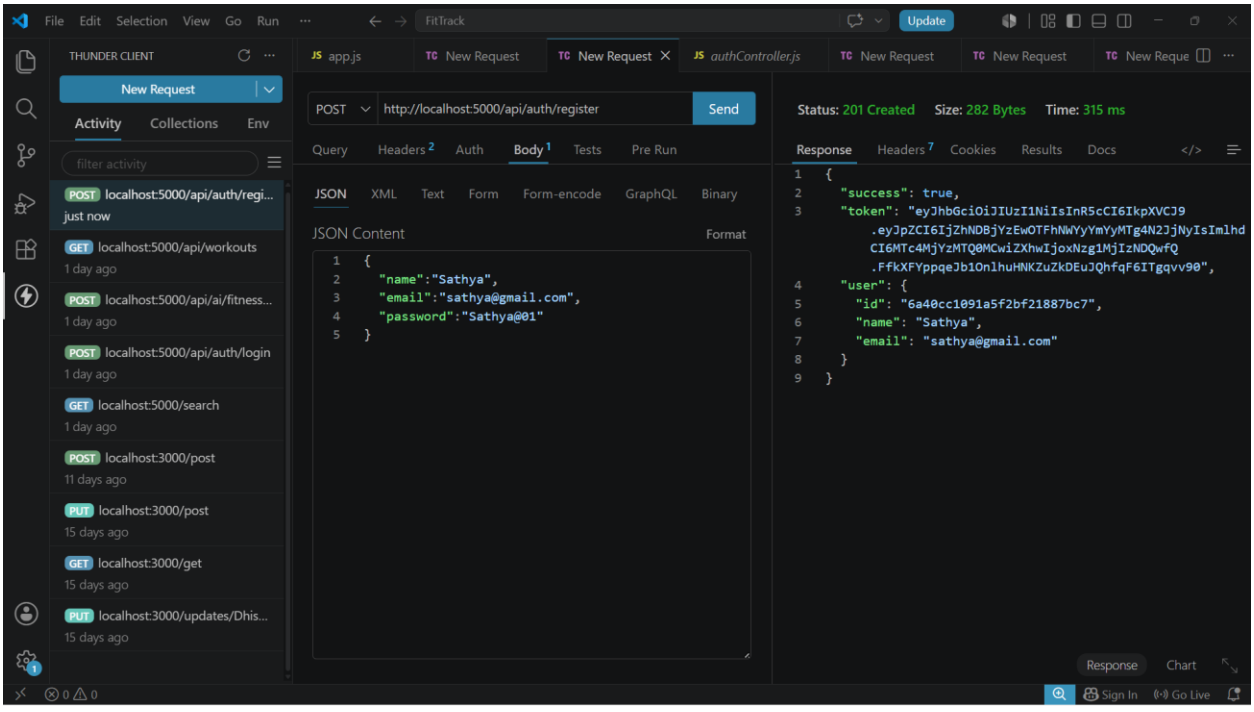
API Testing

Use Postman or Thunder Client to verify authentication, workout management and AI endpoints.

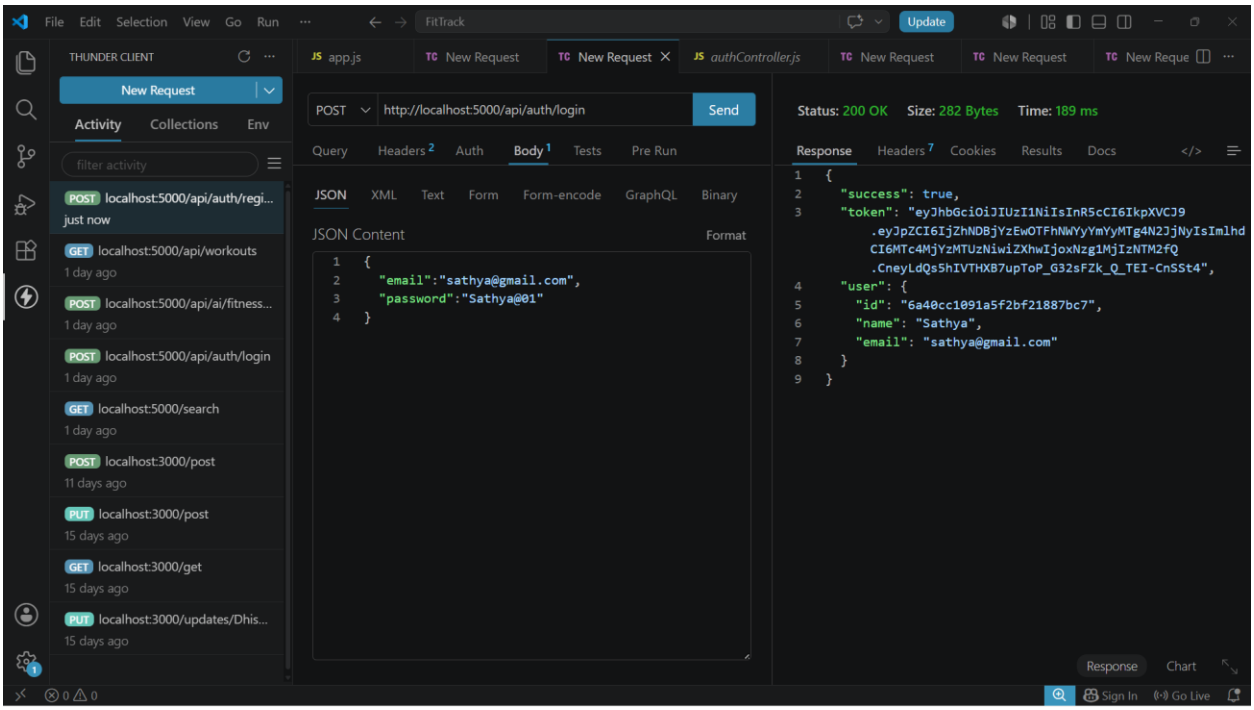
Testing is performed in the following order:

Authentication API Testing

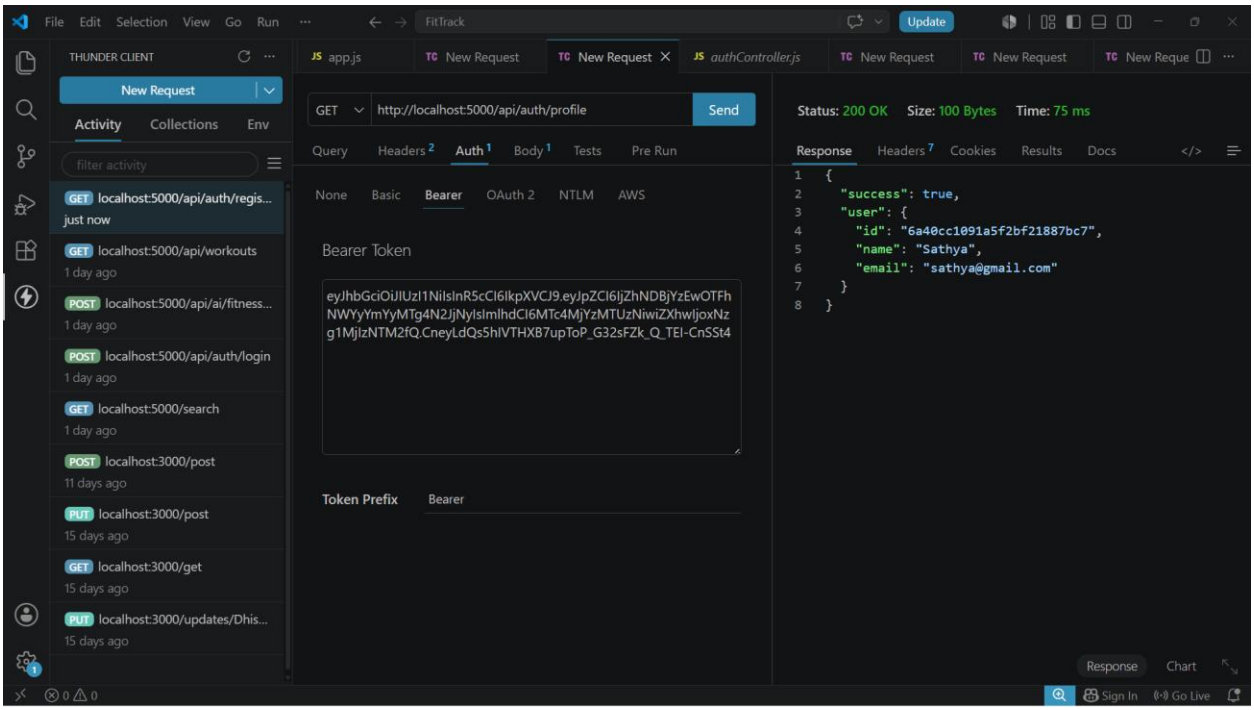
Register User



Login User

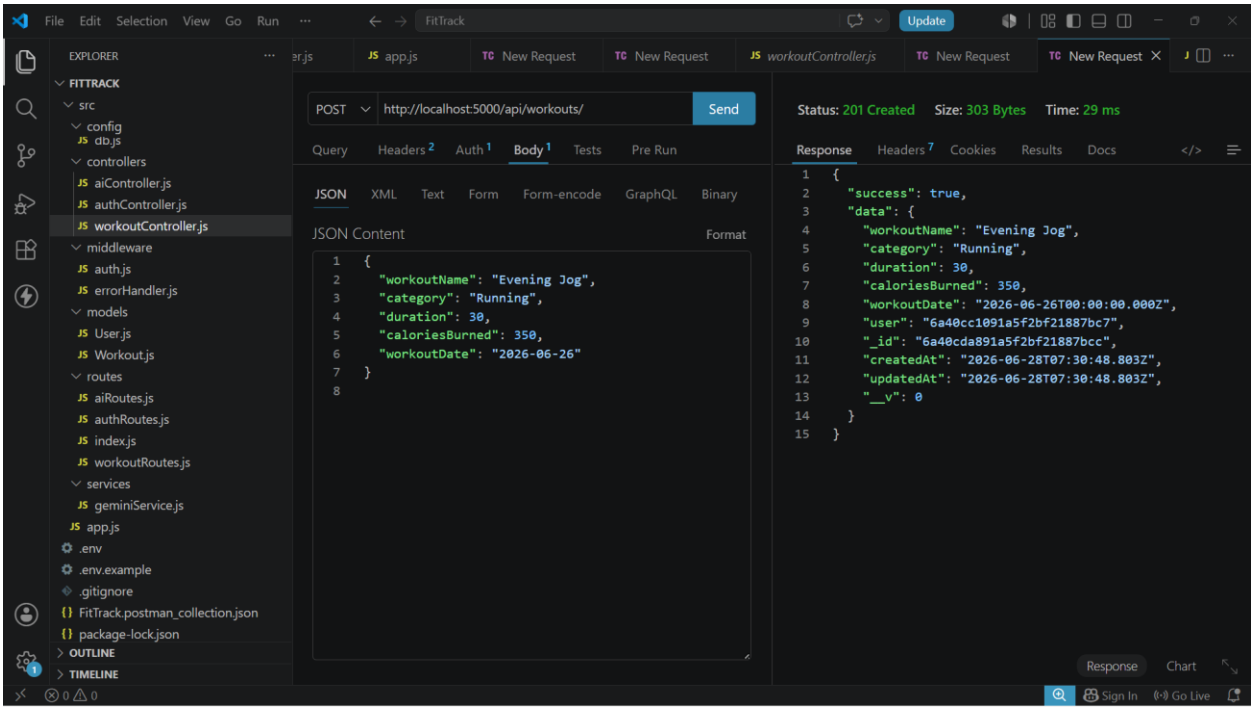


Get Profile

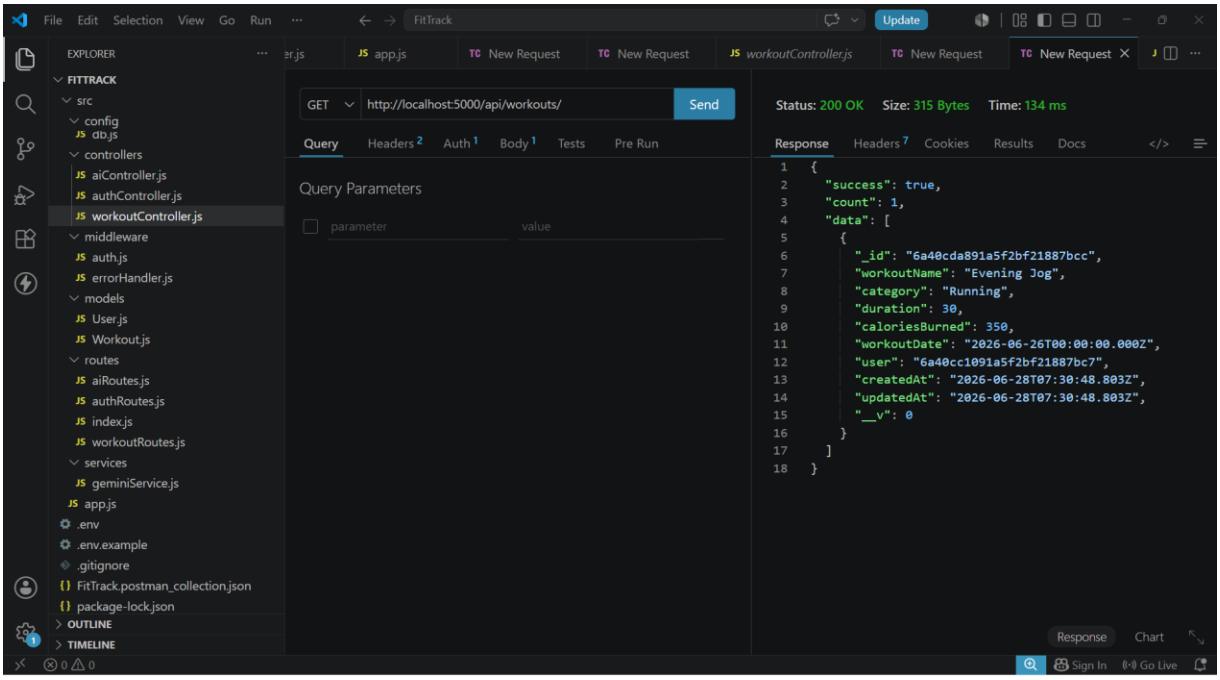


Workout API Testing

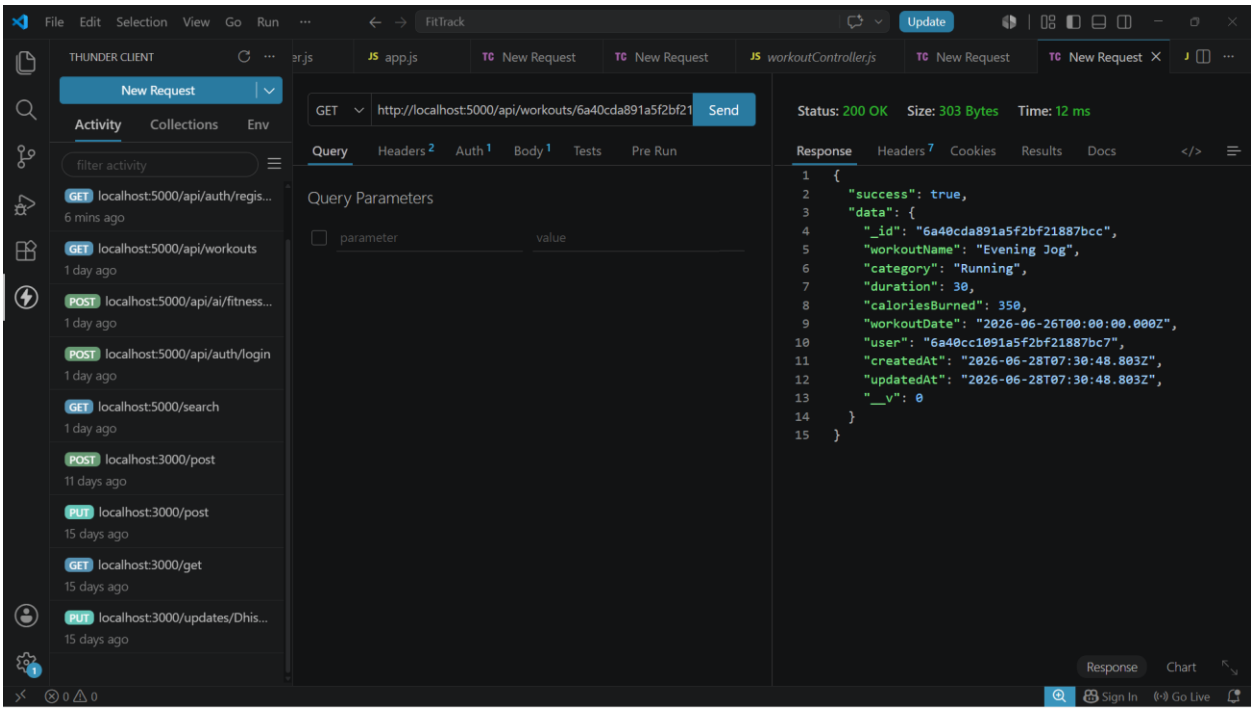
Add Workout



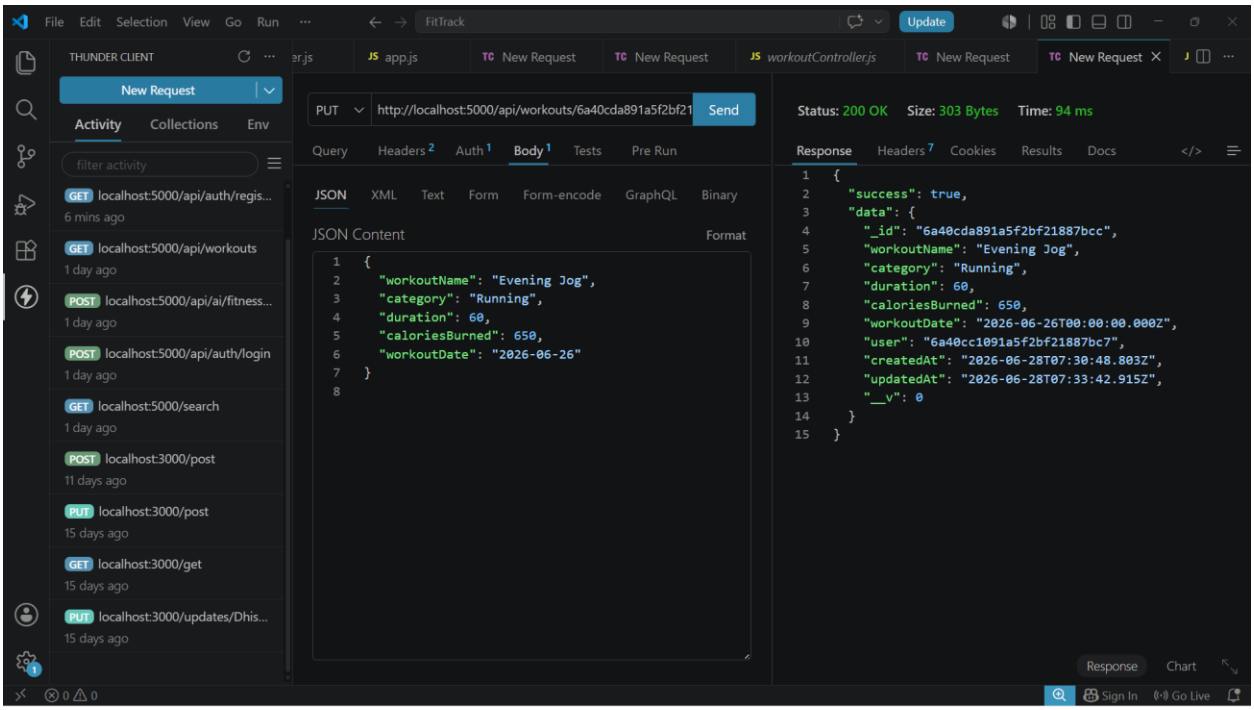
Get All Workouts



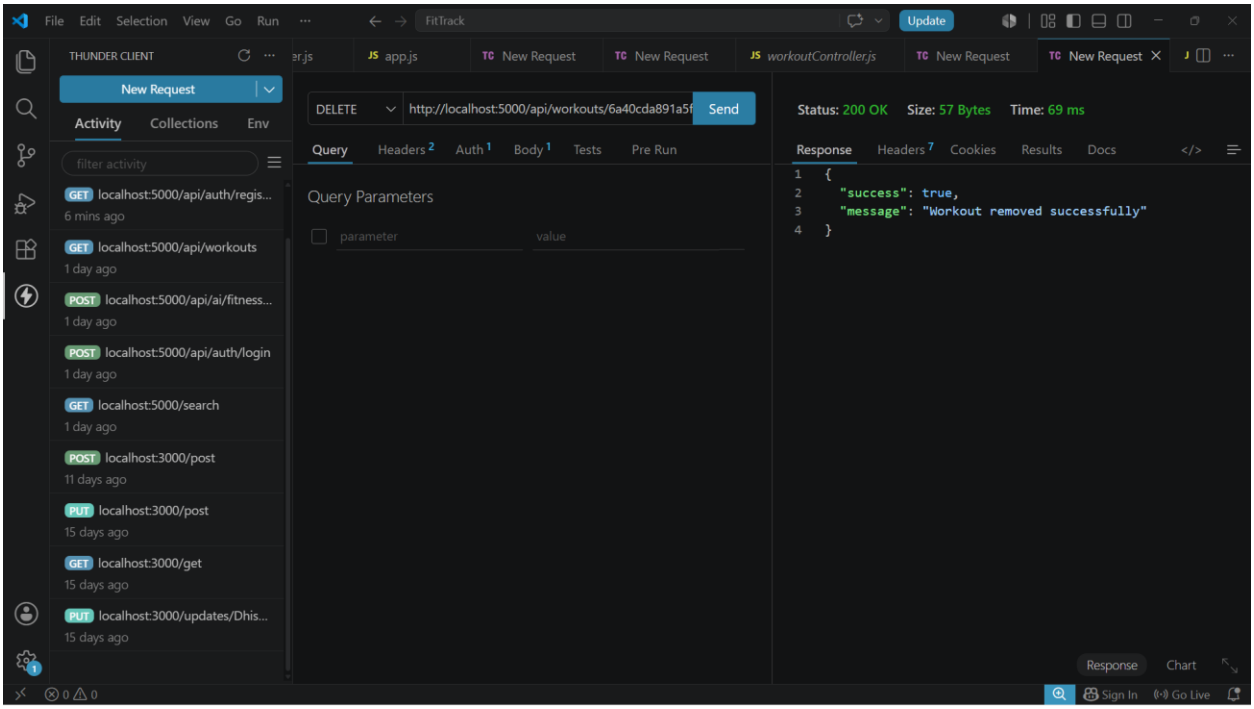
Get Workout By ID



Update Workout

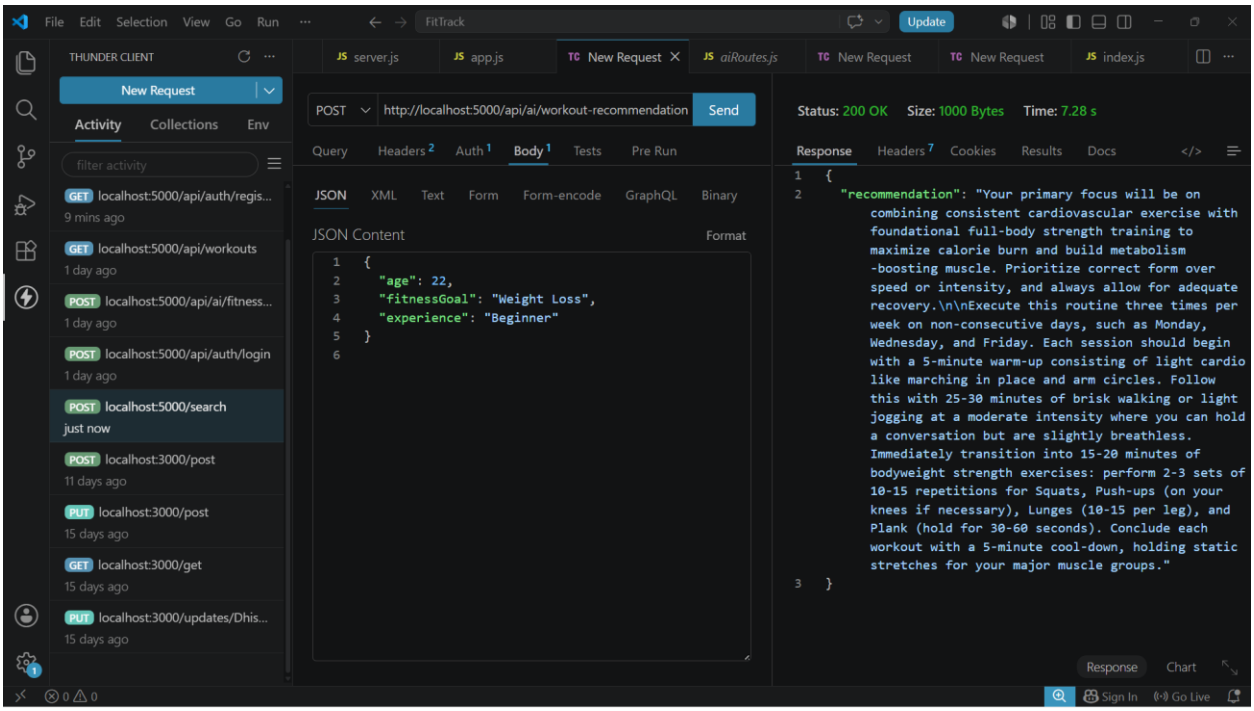


Delete Workout

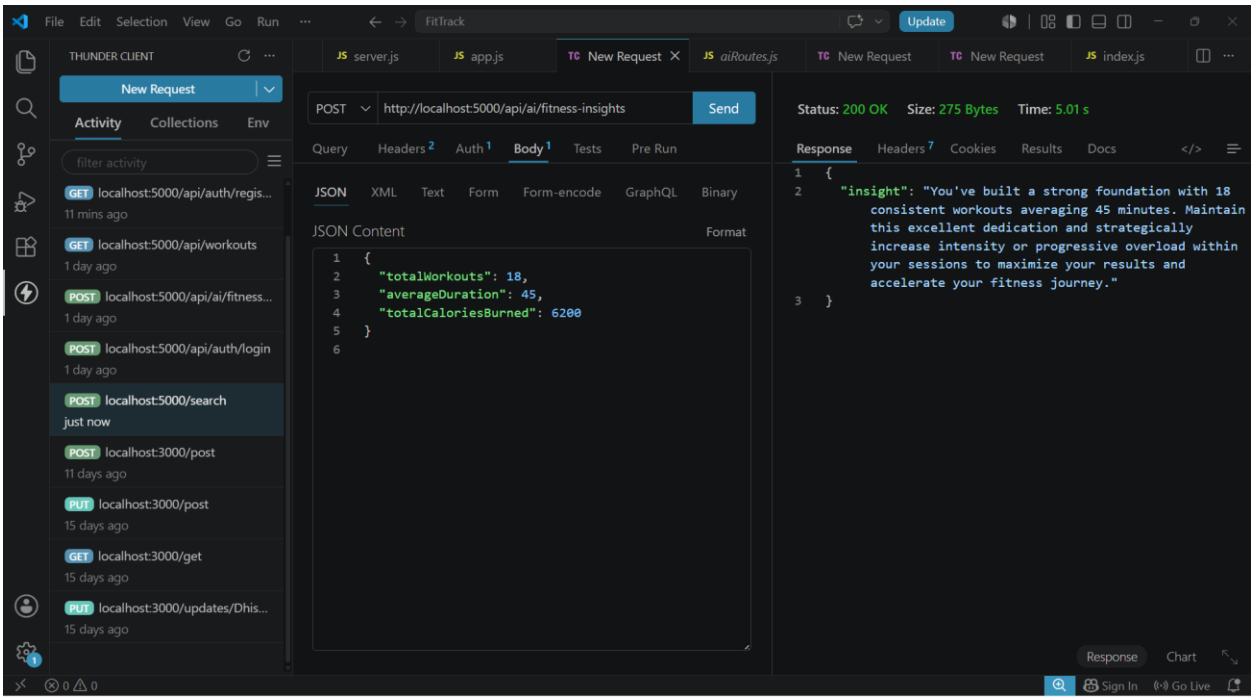


AI API Testing

Workout Recommendation



Fitness Insights



Demo video link:

https://drive.google.com/drive/folders/1vmpVoIapOuIW4wGY-P1bnIV_OJqLNxVY?usp=drive_link

Code files:

https://drive.google.com/drive/folders/1YDq2Uluv2RAj3nMnvFVuVX_OpPmmCWtV?usp=drive_link

Code repository link:

https://github.com/tanish6106/AI-FitTrack.git